DR. RVR NRI INSTITUTE OF TECHNOLOGY

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

REVISED CONSOLIDATED REPORT

10 Case Studies • Algorithms • Flowcharts • Python Programs • Testing

Student Name	A. Balaji
Roll Number	26001374
Branch / Section	CSD-1 (Sec-9)
Course	Python Programming
Academic Year	2026–2027

Prepared in accordance with the Python Programming Activity-Based Learning circular.

CONTENTS

Each case study uses a consistent sequence: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, source code, test cases, sample run and conclusion.

No.	Case study	Main concepts
1	Library Fine Calculator	if-elif-else; cumulative slabs
2	Shopping Bill with Discount	conditions; percentages; currency formatting
3	Exam Result and Grade Report	lists; averages; nested conditions
4	Electricity Bill Estimator	slab calculations; validation
5	Word and Character Analyzer	strings; loops; counting
6	Prime Numbers in a Range	nested loops; modulo; square root
7	Inventory Reorder Alert	tuples; accumulation; continue
8	Employee Shift Overlap Checker	nested loops; interval overlap
9	Student Attendance Eligibility	percentage; while loop
10	Text Run-Length Compression	strings; run-length counting; comparison

Testing note

The test tables include normal, boundary and invalid-input scenarios where relevant. Run the programs in Python before submission and confirm the outputs.

CASE STUDY 01 — Library Fine Calculator

1. Problem Statement

Calculate a library fine from the number of overdue days. Charge Rs. 2 per day for the first 5 days, Rs. 5 per day for days 6–10, and Rs. 10 per day after day 10. Books returned on time have no fine.

2. Objectives

• Apply conditional statements to select a rate.
• Calculate a cumulative fine across multiple slabs.
• Reject negative day counts.

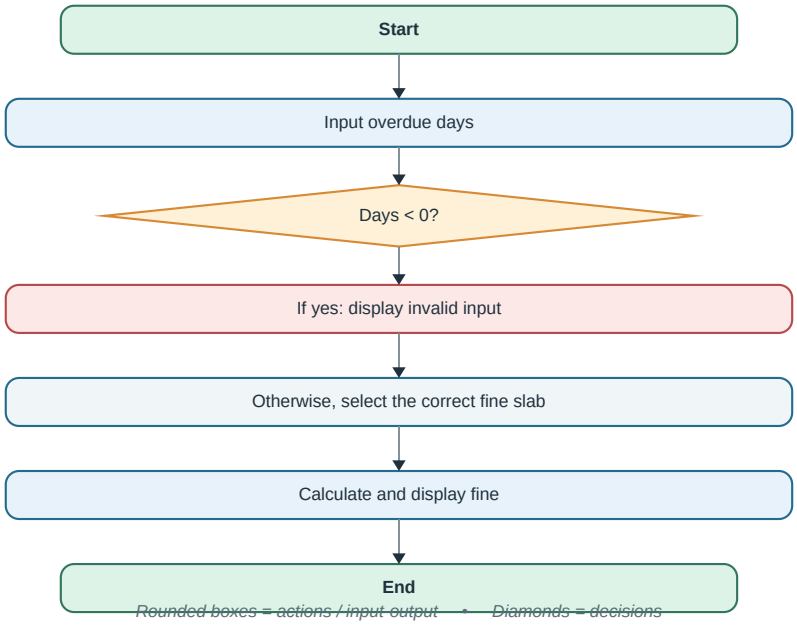
3. Assumptions

• The input is a whole number of days.
• The fine is cumulative: each rate applies only to its own slab.

4. Algorithm

1. Read overdue days.
2. If days are negative, show an error.
3. If days are 0, fine is zero.
4. For 1–5 days, charge days × 2.
5. For 6–10 days, charge 10 + (days – 5) × 5.
6. For more than 10 days, charge 35 + (days – 10) × 10.
7. Display the fine.

5. Flowchart



CASE STUDY 01 — Library Fine Calculator (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT days
03. IF days < 0: PRINT invalid
04. ELSE IF days == 0: fine = 0
05. ELSE IF days <= 5: fine = days * 2
06. ELSE IF days <= 10: fine = 10 + (days - 5) * 5
07. ELSE: fine = 35 + (days - 10) * 10
08. PRINT fine
09. END
```

7. Python Source Code

```
days = int(input("Enter overdue days: "))

if days < 0:
    print("Invalid number of days")
elif days == 0:
    fine = 0
elif days <= 5:
    fine = days * 2
elif days <= 10:
    fine = 5 * 2 + (days - 5) * 5
else:
    fine = 5 * 2 + 5 * 5 + (days - 10) * 10

if days >= 0:
    print(f"Library fine: Rs. {fine}")
```

8. Test Cases

Input / scenario	Expected result	Purpose
8	Rs. 25	Middle slab: 10 + 3 × 5
0	Rs. 0	Returned on time
12	Rs. 55	10 + 25 + 2 × 10
-2	Invalid number of days	Negative input rejected

9. Sample Run

```
INPUT
Enter overdue days: 8

OUTPUT
Library fine: Rs. 25
```

10. Conclusion

The program uses if-elif-else conditions to calculate a cumulative fine and validates negative input.

CASE STUDY 02 — Shopping Bill with Discount

1. Problem Statement

Accept the total shopping amount. Give a 5% discount for amounts from Rs. 1,000 to Rs. 2,999.99 and a 10% discount for Rs. 3,000 or more. Otherwise, no discount is given. Display the discount and final bill.

2. Objectives

• Use conditional logic to choose a discount rate.
• Calculate discount and net bill.
• Format currency values to two decimal places.

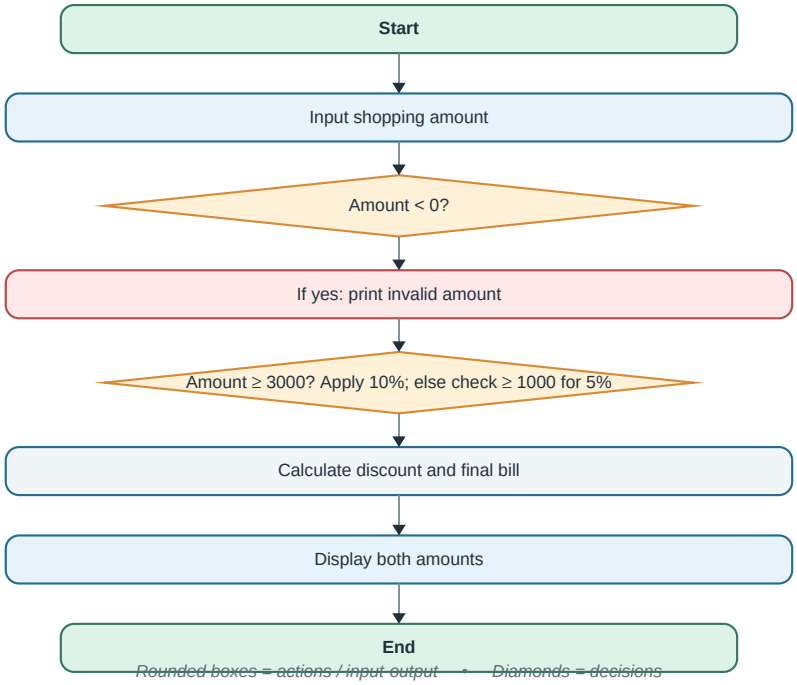
3. Assumptions

• The amount is entered as a non-negative number.
• Discount is applied to the full bill amount.

4. Algorithm

1. Read shopping amount.
2. If amount is negative, display an error.
3. Choose 10% for amount ≥ 3000, 5% for amount ≥ 1000, otherwise 0%.
4. Calculate discount = amount × rate.
5. Calculate final bill = amount – discount.
6. Display both values to two decimal places.

5. Flowchart



CASE STUDY 02 — Shopping Bill with Discount (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT amount
03. IF amount < 0: PRINT invalid
04. ELSE: choose rate (10%, 5%, or 0%)
05. discount = amount * rate
06. final_bill = amount - discount
07. PRINT discount, final_bill
08. END
```

7. Python Source Code

```
amount = float(input("Enter shopping amount (Rs.): "))

if amount < 0:
    print("Invalid amount")
else:
    if amount >= 3000:
        rate = 0.10
    elif amount >= 1000:
        rate = 0.05
    else:
        rate = 0.0

    discount = amount * rate
    final_bill = amount - discount

    print(f"Discount: Rs. {discount:.2f}")
    print(f"Final bill: Rs. {final_bill:.2f}")
```

8. Test Cases

Input / scenario	Expected result	Purpose
3500	Discount Rs. 350.00; final Rs. 3150.00	10% discount
1500	Discount Rs. 75.00; final Rs. 1425.00	5% discount
700	Discount Rs. 0.00; final Rs. 700.00	No discount
-10	Invalid amount	Negative input rejected

9. Sample Run

```
INPUT
Enter shopping amount (Rs.): 3500

OUTPUT
Discount: Rs. 350.00
Final bill: Rs. 3150.00
```

10. Conclusion

The program selects the appropriate discount rate and computes the amount payable.

CASE STUDY 03 — Exam Result and Grade Report

1. Problem Statement

Read marks in three subjects, calculate the average, and display Pass/Fail and a grade. A student fails if any subject mark is below 35. Otherwise, assign A for average ≥ 80, B for ≥ 60, C for ≥ 50, and D for the remaining passing averages.

2. Objectives

• Validate marks within the range 0–100.
• Calculate an average using a list.
• Apply pass/fail rules and grade bands.

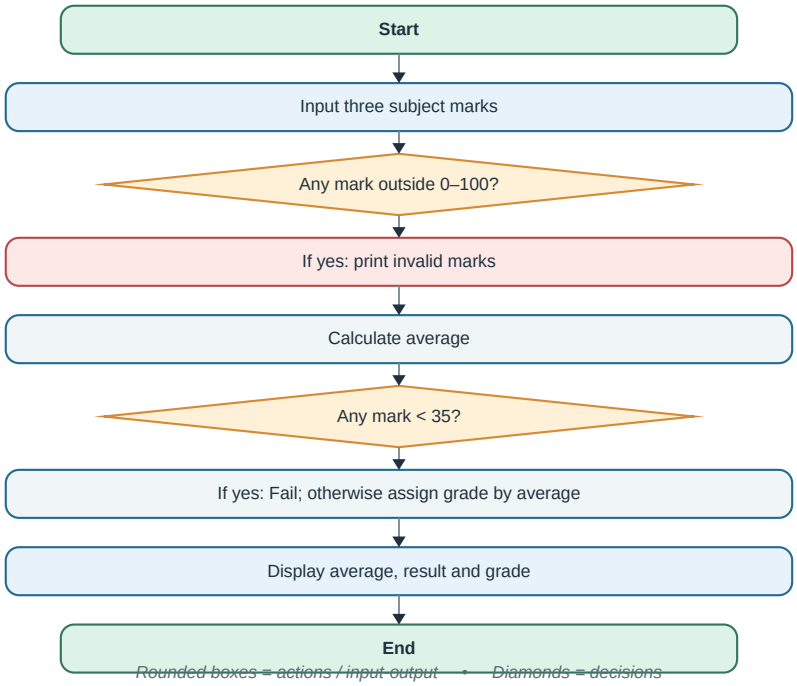
3. Assumptions

• Exactly three subject marks are entered.
• Each mark may be a decimal value between 0 and 100.

4. Algorithm

1. Read three subject marks.
2. If any mark is outside 0–100, display invalid marks.
3. Calculate average = sum of marks ÷ 3.
4. If any mark is below 35, result is Fail and grade is No grade.
5. Otherwise, assign grade A/B/C/D using average thresholds.
6. Display average, result and grade.

5. Flowchart



CASE STUDY 03 — Exam Result and Grade Report (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT mark1, mark2, mark3
03. IF any mark outside 0..100: PRINT invalid
04. ELSE calculate average
05. IF any mark < 35: result = Fail; grade = No grade
06. ELSE choose grade using average thresholds
07. PRINT average, result, grade
08. END
```

7. Python Source Code

```
marks = []

for subject in range(1, 4):
    mark = float(input(f"Enter subject {subject} marks: "))
    marks.append(mark)

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks")
else:
    average = sum(marks) / 3

    if any(mark < 35 for mark in marks):
        result = "Fail"
        grade = "No grade"
    else:
        result = "Pass"
        if average >= 80:
            grade = "A"
        elif average >= 60:
            grade = "B"
        elif average >= 50:
            grade = "C"
        else:
            grade = "D"

    print(f"Average: {average:.2f}")
    print("Result:", result)
    print("Grade:", grade)
```

8. Test Cases

Input / scenario	Expected result	Purpose
78, 84, 69	Average 77.00; Pass; Grade B	Passing marks; average in B band
90, 30, 85	Fail; No grade	One subject below 35
105, 70, 80	Invalid marks	Mark above 100

9. Sample Run

```
INPUT
Subject marks: 78, 84, 69

OUTPUT
Average: 77.00
Result: Pass
Grade: B
```

10. Conclusion

The program combines input validation, list processing, average calculation and nested conditions to produce a grade report.

CASE STUDY 04 — Electricity Bill Estimator

1. Problem Statement

Calculate an electricity bill using slab rates: first 100 units at Rs. 2 per unit, next 100 units at Rs. 3 per unit, and units above 200 at Rs. 5 per unit. Add a fixed charge of Rs. 50.

2. Objectives

• Apply slab-based calculations.
• Validate unit consumption.
• Separate energy charges from fixed charges.

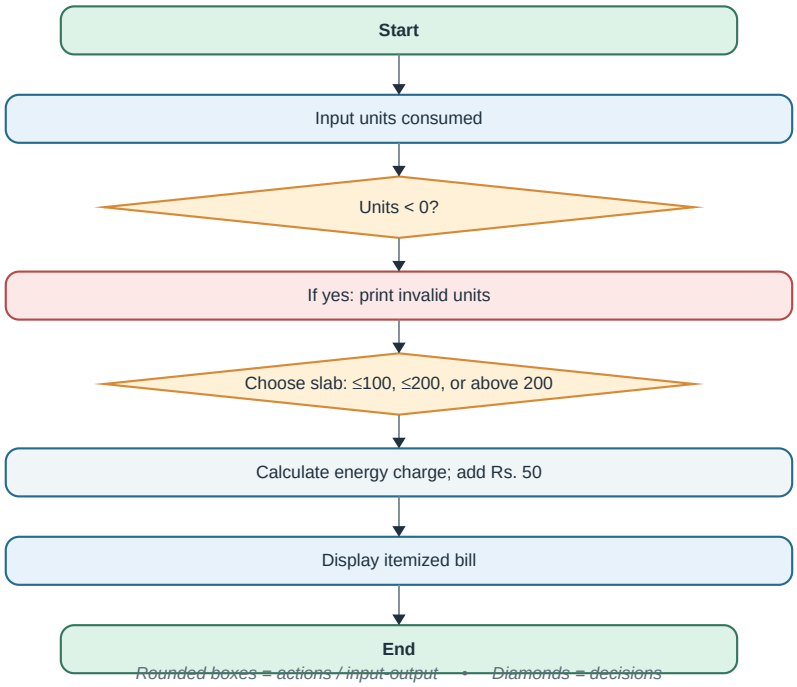
3. Assumptions

• Units consumed cannot be negative.
• The fixed charge is added once to every valid bill.

4. Algorithm

1. Read units consumed.
2. If units are negative, display an error.
3. Calculate energy charge using the appropriate slab(s).
4. Add Rs. 50 fixed charge.
5. Display energy charge, fixed charge and total.

5. Flowchart



CASE STUDY 04 — Electricity Bill Estimator (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT units
03. IF units < 0: PRINT invalid
04. ELSE calculate energy charge by slab
05. total = energy + 50
06. PRINT energy, fixed charge, total
07. END
```

7. Python Source Code

```
units = float(input("Enter units consumed: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed_charge = 50
    total = energy + fixed_charge

    print(f"Energy charge: Rs. {energy:.2f}")
    print(f"Fixed charge: Rs. {fixed_charge:.2f}")
    print(f"Total bill: Rs. {total:.2f}")
```

8. Test Cases

Input / scenario	Expected result	Purpose
250	Energy Rs. 650.00; total Rs. 700.00	All three slabs
80	Energy Rs. 160.00; total Rs. 210.00	First slab
150	Energy Rs. 350.00; total Rs. 400.00	First and second slabs
-1	Invalid units	Negative input rejected

9. Sample Run

```
INPUT
Enter units consumed: 250

OUTPUT
Energy charge: Rs. 650.00
Fixed charge: Rs. 50.00
Total bill: Rs. 700.00
```

10. Conclusion

The program calculates progressive slab charges and adds a fixed charge to obtain the final bill.

CASE STUDY 05 — Word and Character Analyzer

1. Problem Statement

Accept a sentence and report the number of characters (excluding leading and trailing spaces), words, vowels and digits.

2. Objectives

• Use string methods such as strip(), split() and lower().
• Count selected character categories.
• Understand iteration and built-in functions.

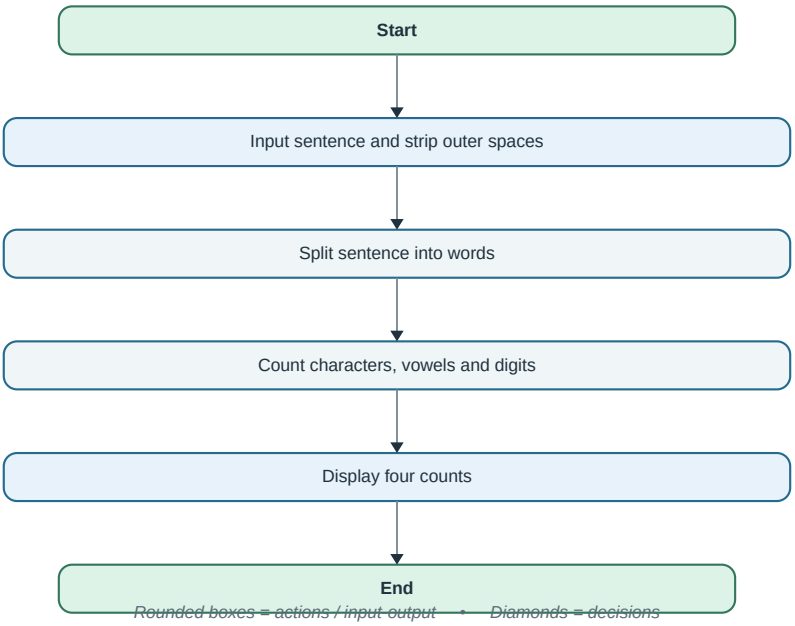
3. Assumptions

• Spaces between words count as characters.
• Only English vowels a, e, i, o and u are counted as vowels.

4. Algorithm

1. Read a sentence and remove outer spaces.
2. Split the cleaned sentence into words.
3. Count characters using len().
4. Count vowels and digits by examining each character.
5. Display all four counts.

5. Flowchart



CASE STUDY 05 — Word and Character Analyzer (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT sentence
03. text = sentence.strip()
04. words = text.split()
05. COUNT characters, vowels and digits
06. PRINT counts
07. END
```

7. Python Source Code

```
text = input("Enter a sentence: ").strip()
words = text.split()

vowels = 0
digits = 0

for ch in text.lower():
    if ch in "aeiou":
        vowels += 1
    if ch.isdigit():
        digits += 1

print("Characters:", len(text))
print("Words:", len(words))
print("Vowels:", vowels)
print("Digits:", digits)
```

8. Test Cases

Input / scenario	Expected result	Purpose
Python 3 is fun	Characters 15; Words 4; Vowels 3; Digits 1	Typical sentence
Hello	Characters 5; Words 1; Vowels 2; Digits 0	Outer spaces removed
	Characters 0; Words 0; Vowels 0; Digits 0	Empty sentence

9. Sample Run

```
INPUT
Enter a sentence: Python 3 is fun

OUTPUT
Characters: 15
Words: 4
Vowels: 3
Digits: 1
```

10. Conclusion

The program demonstrates string cleaning, word splitting and character-by-character counting.

CASE STUDY 06 — Prime Numbers in a Range

1. Problem Statement

Display all prime numbers between two entered integers, inclusive. Validate that the starting number is not greater than the ending number.

2. Objectives

• Use for loops and conditional statements.
• Test divisibility using the modulo operator.
• Reduce divisor checks by testing up to the square root.

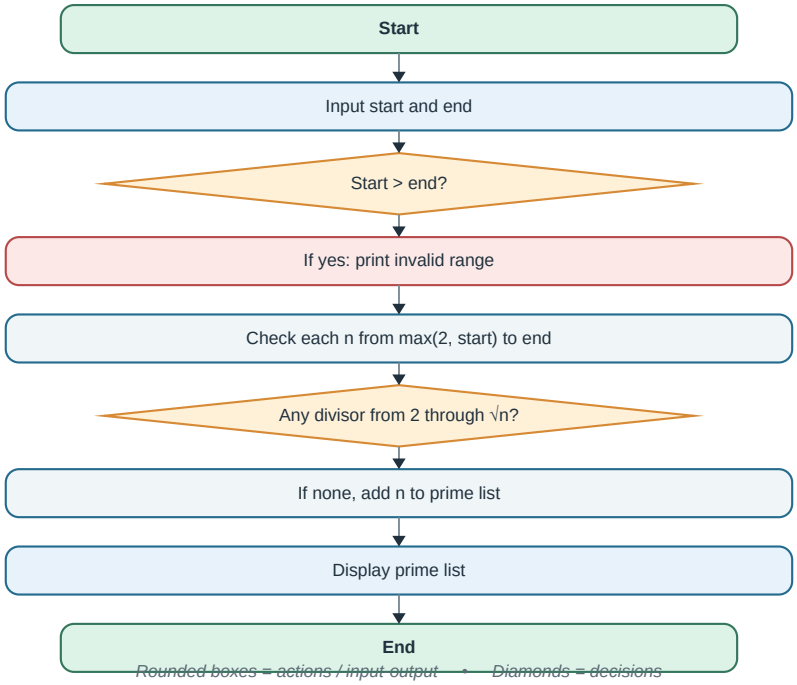
3. Assumptions

• Both range limits are integers.
• Numbers below 2 are not prime.

4. Algorithm

1. Read start and end values.
2. If start > end, display an invalid-range message.
3. Visit each number n from max(2, start) through end.
4. Test divisors from 2 through floor(√n).
5. If no divisor is found, add n to the prime list.
6. Display the list.

5. Flowchart



CASE STUDY 06 — Prime Numbers in a Range (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT start, end
03. IF start > end: PRINT invalid range
04. primes = empty list
05. FOR each n from max(2,start) to end
06. Test divisors d from 2 to √n
07. IF no divisor: add n to primes
08. PRINT primes
09. END
```

7. Python Source Code

```
start = int(input("Enter start: "))
end = int(input("Enter end: "))

if start > end:
    print("Invalid range")
else:
    primes = []

    for n in range(max(2, start), end + 1):
        is_prime = True

        for divisor in range(2, int(n ** 0.5) + 1):
            if n % divisor == 0:
                is_prime = False
                break

        if is_prime:
            primes.append(n)

    print("Prime numbers:", primes)
```

8. Test Cases

Input / scenario	Expected result	Purpose
10, 30	[11, 13, 17, 19, 23, 29]	Normal range
-4, 5	[2, 3, 5]	Range includes negatives
20, 10	Invalid range	Start greater than end

9. Sample Run

```
INPUT
Enter start: 10
Enter end: 30

OUTPUT
Prime numbers: [11, 13, 17, 19, 23, 29]
```

10. Conclusion

The program uses nested loops and an efficient divisor limit to find primes within a validated range.

CASE STUDY 07 — Inventory Reorder Alert

1. Problem Statement

Store products as name, price and stock quantity. Calculate total inventory value and list products with fewer than 5 items in stock. Report and skip records with negative prices or quantities.

2. Objectives

• Represent product records as tuples.
• Use loops to process multiple records.
• Accumulate a total and build a reorder list.

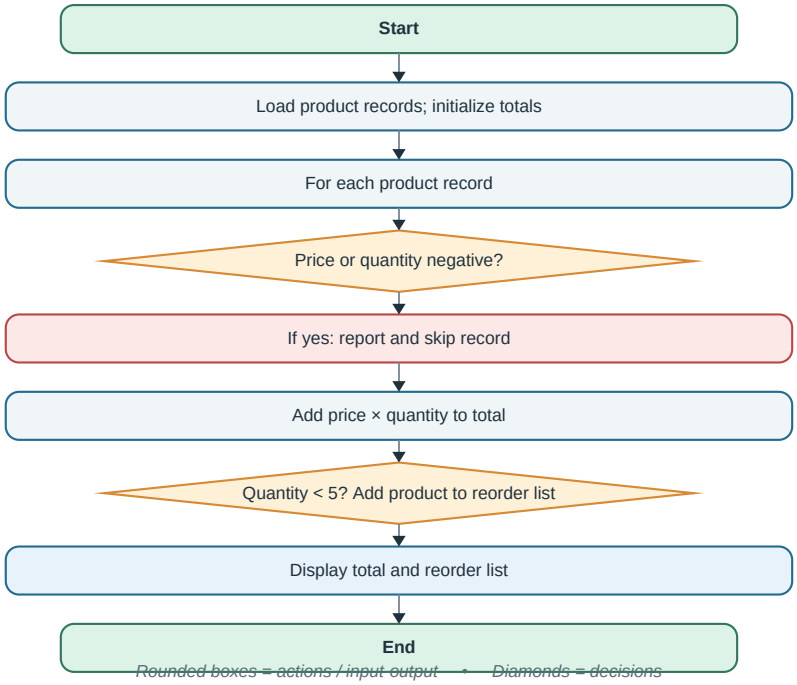
3. Assumptions

• The sample product records are stored in the program.
• A stock quantity below 5 triggers a reorder alert; quantity 5 does not.

4. Algorithm

1. Load product records.
2. Initialize total value to zero and reorder list to empty.
3. For each product, reject negative price or quantity.
4. For each valid record, add price × quantity to total.
5. If quantity < 5, add the product name to reorder list.
6. Display total value and reorder list.

5. Flowchart



CASE STUDY 07 — Inventory Reorder Alert (CONTINUED)

6. Pseudocode

```
01. START
02. LOAD product records
03. total = 0; reorder = []
04. FOR each product
05. IF price < 0 OR quantity < 0: report and skip
06. total = total + price * quantity
07. IF quantity < 5: add name to reorder
08. PRINT total and reorder list
09. END
```

7. Python Source Code

```
products = [
    ("Pen", 10, 3),
    ("Notebook", 50, 12),
    ("Marker", 25, 2),
    ("Folder", 30, 6)
]

total_value = 0
reorder = []

for name, price, quantity in products:
    if price < 0 or quantity < 0:
        print("Invalid record:", name)
        continue

    total_value += price * quantity

    if quantity < 5:
        reorder.append(name)

print(f"Total inventory value: Rs. {total_value:.2f}")
print("Reorder products:", reorder if reorder else "None")
```

8. Test Cases

Input / scenario	Expected result	Purpose
Provided sample records	Total Rs. 1070.00; reorder ['Pen', 'Marker']	Normal sample
Quantity = 5	Product not added to reorder list	Threshold boundary
Negative price or quantity	Invalid record reported and skipped	Invalid record

9. Sample Run

```
INPUT
Sample product records are included in the program.

OUTPUT
Total inventory value: Rs. 1070.00
Reorder products: ['Pen', 'Marker']
```

10. Conclusion

The program processes structured product records, calculates stock value and identifies low-stock items.

CASE STUDY 08 — Employee Shift Overlap Checker

1. Problem Statement

Given employee shifts as (employee name, start hour, end hour), identify pairs of shifts for the same employee that overlap. Valid times satisfy 0 ≤ start < end ≤ 24.

2. Objectives

• Validate time intervals.
• Compare pairs using nested loops.
• Detect overlap using interval boundaries.

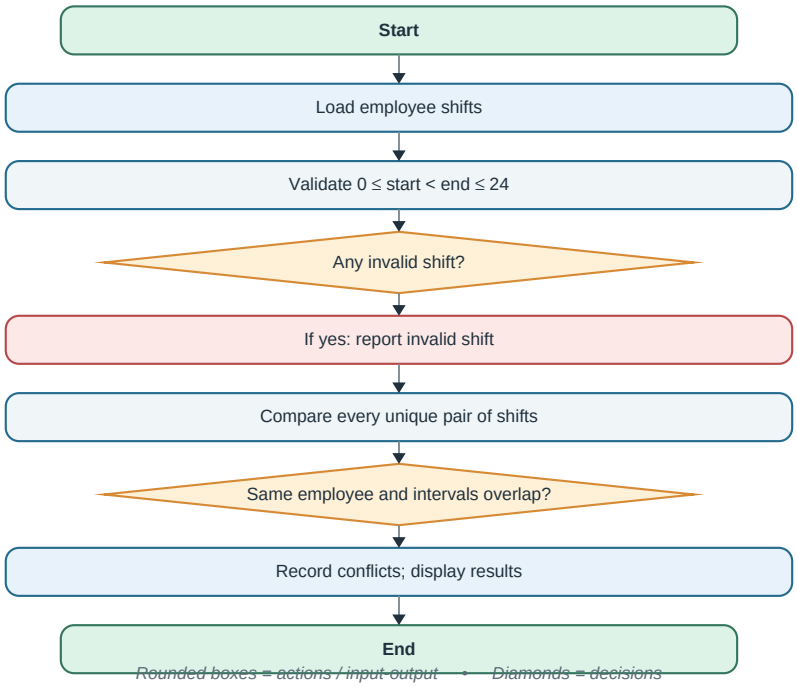
3. Assumptions

• Shift end time equal to another shift's start time is not an overlap.
• Sample shifts are stored as tuples in the program.

4. Algorithm

1. Load shifts and validate every time interval.
2. If any shift is invalid, report invalid records and do not compare shifts.
3. Compare every unique pair of shifts.
4. A conflict exists when names match and max(start times) < min(end times).
5. Display each conflict or a no-conflict message.

5. Flowchart



CASE STUDY 08 — Employee Shift Overlap Checker (CONTINUED)

6. Pseudocode

```
01. START
02. LOAD shifts
03. VALIDATE each shift: 0 ≤ start < end ≤ 24
04. IF any invalid: report invalid shift
05. ELSE compare every unique pair (i, j)
06. IF same employee AND intervals overlap: record conflict
07. PRINT conflicts or no-conflict message
08. END
```

7. Python Source Code

```
shifts = [
    ("Ravi", 9, 13),
    ("Ravi", 12, 16),
    ("Maya", 10, 14),
    ("Ravi", 16, 18)
]

valid = True
for name, start, end in shifts:
    if not (0 <= start < end <= 24):
        print("Invalid shift:", (name, start, end))
        valid = False

if valid:
    conflicts = []

    for i in range(len(shifts)):
        for j in range(i + 1, len(shifts)):
            first = shifts[i]
            second = shifts[j]

            same_employee = first[0] == second[0]
            overlap = max(first[1], second[1]) < min(first[2], second[2])

            if same_employee and overlap:
                conflicts.append((first, second))

    if conflicts:
        for first, second in conflicts:
            print("Conflict:", first, "and", second)
    else:
        print("All shifts are valid; no conflicts")
```

8. Test Cases

Input / scenario	Expected result	Purpose
Ravi 9–13 and Ravi 12–16	Conflict detected	Intervals overlap from 12 to 13
Ravi 9–13 and Ravi 13–16	No conflict	Touching endpoints are not overlap
Start 25, end 26	Invalid shift	Outside 0–24

9. Sample Run

```
INPUT
Sample shifts are included in the program.

OUTPUT
Conflict: ('Ravi', 9, 13) and ('Ravi', 12, 16)
```

10. Conclusion

The program validates shift times and uses pairwise interval comparison to identify scheduling conflicts.

CASE STUDY 09 — Student Attendance Eligibility

1. Problem Statement

Read classes attended and total classes. Calculate attendance percentage and determine whether the student meets a 75% requirement. If below 75%, calculate the minimum number of consecutive classes the student must attend, with no absences, to reach 75%.

2. Objectives

• Validate attendance counts.
• Calculate percentages.
• Use a while loop to find the minimum additional classes required.

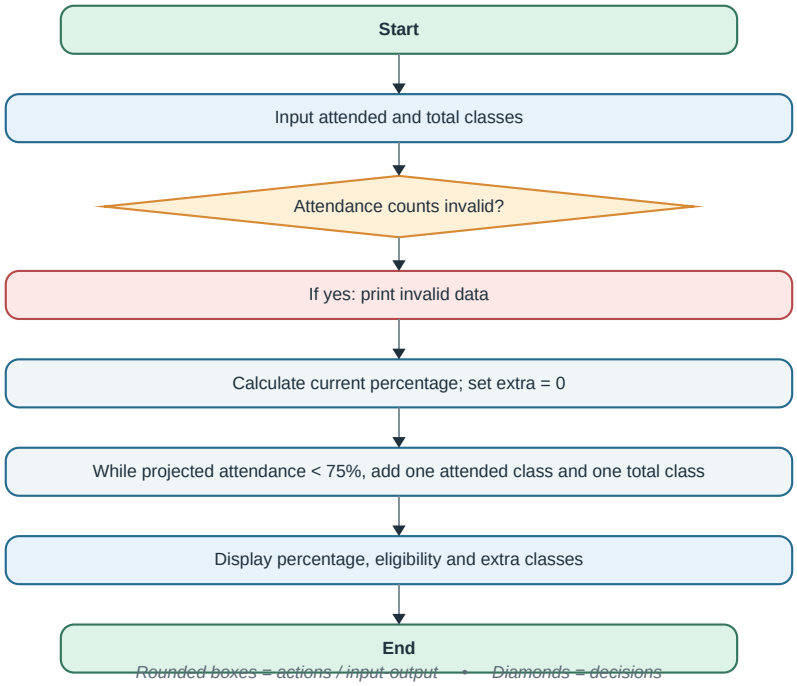
3. Assumptions

• Total classes must be greater than zero.
• Every future class attended increases both attended and total classes by one.

4. Algorithm

1. Read attended and total class counts.
2. Reject total ≤ 0, attended < 0 or attended > total.
3. Calculate current attendance percentage.
4. Initialize extra classes to zero.
5. While (attended + extra) / (total + extra) is below 75%, increase extra by one.
6. Display current percentage and eligibility; display extra classes if needed.

5. Flowchart



CASE STUDY 09 — Student Attendance Eligibility (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT attended, total
03. IF total ≤ 0 OR attended < 0 OR attended > total: PRINT invalid
04. percent = attended / total × 100
05. extra = 0
06. WHILE (attended + extra)/(total + extra) < 0.75: extra += 1
07. PRINT percent and eligibility
08. IF extra > 0: PRINT extra classes
09. END
```

7. Python Source Code

```
attended = int(input("Classes attended: "))
total = int(input("Total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data")
else:
    percent = attended / total * 100
    extra = 0

    while (attended + extra) / (total + extra) * 100 < 75:
        extra += 1

    print(f"Current attendance: {percent:.2f}%")

    if extra == 0:
        print("Eligibility: Meets 75% requirement")
    else:
        print("Eligibility: Below 75%")
        print("Extra classes to attend:", extra)
```

8. Test Cases

Input / scenario	Expected result	Purpose
30 attended, 40 total	75.00%; Meets requirement	Exactly at threshold
20 attended, 40 total	50.00%; 40 extra classes	Below threshold
41 attended, 40 total	Invalid attendance data	Attended exceeds total

9. Sample Run

```
INPUT
Classes attended: 30
Total classes: 40

OUTPUT
Current attendance: 75.00%
Eligibility: Meets 75% requirement
```

10. Conclusion

The program validates attendance data and uses repetition to find the smallest number of additional attended classes needed.

CASE STUDY 10 — Text Run-Length Compression

1. Problem Statement

Compress consecutive repeated characters by writing each character followed by its count (for example, aaabb becomes a3b2). Display the encoded text and choose it only when it is shorter than the original.

2. Objectives

• Use while loops to process character runs.
• Build a new string incrementally.
• Compare string lengths to choose a representation.

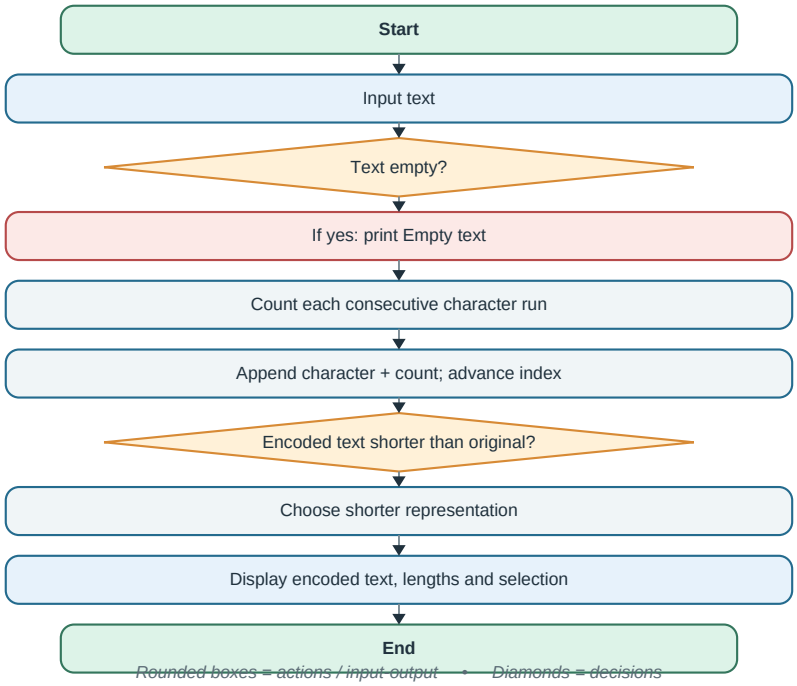
3. Assumptions

• Compression is case-sensitive and preserves spaces and punctuation as characters.
• The encoding is a simple teaching example; characters that are digits can make decoding ambiguous.

4. Algorithm

1. Read text.
2. If text is empty, display an empty-text message.
3. Set encoded text to an empty string and index to zero.
4. Count consecutive matching characters from the current index.
5. Append the character and its count to encoded text; move index past the run.
6. Compare encoded and original lengths.
7. Display encoded text, lengths and selected representation.

5. Flowchart



CASE STUDY 10 — Text Run-Length Compression (CONTINUED)

6. Pseudocode

```
01. START
02. INPUT text
03. IF text is empty: PRINT Empty text
04. ELSE encoded = ''; i = 0
05. WHILE i < length(text): count matching run
06. Append character + count; advance i by count
07. IF length(encoded) < length(text): selected = encoded
08. ELSE selected = original text
09. PRINT encoded, lengths, selected
10. END
```

7. Python Source Code

```
text = input("Enter text: ")

if not text:
    print("Empty text")
else:
    encoded = ""
    i = 0

    while i < len(text):
        character = text[i]
        count = 1

        while (i + count < len(text)
               and text[i + count] == character):
            count += 1

        encoded += character + str(count)
        i += count

    selected = encoded if len(encoded) < len(text) else text

    print("Encoded text:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))
    print("Selected representation:", selected)
```

8. Test Cases

Input / scenario	Expected result	Purpose
aaabbcccc	Encoded a3b2c4; selected a3b2c4	Compression is shorter
abc	Encoded a1b1c1; selected abc	Original is shorter
(empty input)	Empty text	Empty input handled

9. Sample Run

```
INPUT
Enter text: aaabbcccc

OUTPUT
Encoded text: a3b2c4
Original length: 9
Encoded length: 6
Selected representation: a3b2c4
```

10. Conclusion

The program identifies consecutive character runs and chooses the shorter representation based on string length.